

Studio NST

Neural Style Transfer & Visual Experimentation

Technical Report

A research-oriented implementation and experimental evaluation of three neural style transfer approaches.

Experimental environment: NVIDIA GeForce GTX 1650 · 4 GB VRAM · CUDA 13.0 · PyTorch 2.14.0+cu130 · Python 3.14

1. Abstract

Studio NST is a research-oriented implementation of neural style transfer that brings three distinct approaches into one reproducible experimental environment: optimization-based style transfer inspired by Gatys et al., feed-forward style transfer based on the Johnson et al. approach, and arbitrary style transfer using Adaptive Instance Normalization (AdaIN).

The project evaluates computational behavior through runtime, GPU memory, optimization losses, and visual inspection. Experiments were performed on a consumer NVIDIA GeForce GTX 1650 with 4 GB of GPU memory.

The results demonstrate the different computational characteristics of iterative optimization, feed-forward transformation, and feature-statistics-based arbitrary style transfer. The Johnson experiment is explicitly treated as a pipeline validation because its limited training setup did not produce a sufficiently trained model for a meaningful quality comparison.

2. Research Motivation

Neural style transfer demonstrates how deep visual representations can be used to separate image content from artistic style and recombine them into a new image.

Studio NST studies three approaches representing different computational strategies. Gatys performs direct optimization of the generated image. Johnson learns a transformation network that can perform feed-forward inference. AdaIN aligns feature statistics between content and style representations and uses a decoder to reconstruct the stylized image.

3. Research Questions

- How does optimization-based style transfer behave as the number of optimization steps increases?
- How does input resolution affect runtime and GPU memory usage?
- What practical differences exist between iterative optimization and feed-forward transformation?
- How does AdaIN enable arbitrary style transfer?
- How does AdaIN style strength affect the interpolation between content and stylized representations?
- What are the practical constraints of neural style transfer on a 4 GB GPU?

4. Methods

4.1 Gatys

The Gatys implementation directly optimizes the generated image. A pretrained VGG19 network provides intermediate feature representations. Content similarity is represented through VGG features, while style is represented using Gram matrices computed from multiple feature layers.

The implementation exposes configurable optimization steps, content weight, style weight, and learning rate.

4.2 Johnson

The Johnson implementation uses a feed-forward transformation network containing convolutional feature extraction, downsampling layers, residual blocks, and upsampling layers. Training uses perceptual losses derived from VGG19 features.

The current experiment uses a limited 200-image CIFAR-10 subset to validate the training and inference pipeline. It is not treated as a production-quality training configuration.

4.3 AdaIN

Adaptive Instance Normalization aligns the channel-wise mean and standard deviation of content features with those of style features. The resulting representation is passed through a pretrained decoder.

This allows arbitrary style images to be used without training a separate transformation network for each style.

5. Experimental Setup

Component	Configuration
GPU	NVIDIA GeForce GTX 1650
GPU Memory	4 GB
CUDA Runtime	13.0
PyTorch	2.14.0+cu130
Python	3.14
Operating System	Windows

Runtime was measured using Python's high-resolution performance counter. CUDA synchronization was used where required around GPU operations. Peak GPU memory was measured using PyTorch CUDA memory statistics.

6. Gatys Step Scaling

The same content and style image pair was evaluated at a maximum size of 256 pixels using 100, 300, and 500 optimization steps.

Steps	Runtime	Peak GPU Memory	Content Loss	Style Loss
100	5.0686 s	264.57 MB	10.795232	7.127049e-06
300	13.5208 s	263.63 MB	9.615572	5.248773e-06
500	22.5941 s	263.63 MB	9.292706	4.966042e-06

Increasing optimization steps increased runtime while continuing to reduce the recorded optimization losses. The improvement in content loss became smaller as additional optimization steps were added.

7. Gatys Resolution Scaling

The same image pair was evaluated at 256, 384, and 512 pixel maximum resolutions using 300 optimization steps.

Resolution	Runtime	Peak GPU Memory	Content Loss	Style Loss
256 px	14.1449 s	264.57 MB	9.591365	5.511013e-06
384 px	29.0578 s	451.32 MB	6.550163	2.810368e-06
512 px	53.6524 s	729.36 MB	5.367336	2.035976e-06

Increasing resolution substantially increased runtime and peak GPU memory in the recorded experiment. The recorded losses also decreased with resolution, but the losses alone do not establish that higher resolution produced better perceived image quality.

8. Johnson Pipeline Validation

The Johnson experiment used a limited 200-image CIFAR-10 subset and 50 training steps. The complete training and inference pipeline executed successfully.

Measurement	Initial	Final
Total Loss	325.259888	101.421913
Content Loss	9.104837	8.196975
Style Loss	3.161550e-03	9.322494e-04

Training runtime: 6.15 seconds.

Although the pipeline completed successfully, the generated image did not provide satisfactory style transfer quality. The experiment is therefore treated as pipeline validation rather than evidence of a sufficiently trained feed-forward style transfer model.

9. AdaIN Inference

Measurement	Result
Resolution	512 px maximum
Alpha	1.0
Runtime	0.8758 s
Peak GPU Memory	147.68 MB
Device	NVIDIA GeForce GTX 1650

The generated result preserved the main structure of the content image while transferring visible characteristics of the style image.

10. AdaIN Style Strength Experiment

The same content and style image pair was evaluated at a maximum size of 512 pixels using five AdaIN alpha values. Alpha controls the strength of the style-transfer interpolation.

Alpha	Runtime	Peak GPU Memory
0.00	0.4021 s	147.68 MB
0.25	0.2863 s	147.68 MB
0.50	0.2627 s	147.68 MB
0.75	0.2944 s	147.68 MB
1.00	0.2566 s	147.68 MB

Peak GPU memory remained constant at 147.68 MB across the tested alpha values. Runtime varied between 0.2566 and 0.4021 seconds across the individual runs. Because each alpha value was measured once, these runtime differences should not be interpreted as evidence that alpha itself changes inference speed.

11. Evaluation

Studio NST uses runtime, peak GPU memory, optimization losses, and visual inspection as evaluation dimensions.

Optimization losses describe the mathematical objective used during optimization and are not direct perceptual quality scores. Visual inspection is therefore reported separately from numerical measurements.

12. Limitations

- The primary visual evaluation uses one content image and one style image.
- The Johnson experiment uses only 200 CIFAR-10 images for pipeline validation.
- The Johnson model was not trained sufficiently for a meaningful quality comparison.
- Optimization losses are not direct measures of human-perceived image quality.
- Experiments were performed on a single GTX 1650 with 4 GB of GPU memory.
- Runtime and memory measurements can vary across hardware and system conditions.
- The AdaIN alpha experiment uses one content/style pair and one run per alpha value.
- Video style transfer and temporal consistency were not evaluated.

13. Reproducibility

```
python scripts/run_gatys.py --content assets/content/content1.jpg --style assets/styles/style1.jpg
--steps 500 --output outputs/gatys/

python scripts/benchmark.py --content assets/content/content1.jpg --style assets/styles/style1.jpg
--max-size 256

python scripts/benchmark_resolution.py --content assets/content/content1.jpg --style
assets/styles/style1.jpg

python scripts/run_adain.py --content assets/content/content1.jpg --style assets/styles/style1.jpg
--output outputs/adain/stylized.png --max-size 512
```

14. Architecture

The repository separates the three methods into independent implementations while sharing image processing, model utilities, experiment scripts, output handling, and measurement infrastructure.

The detailed system architecture is documented in docs/architecture/system-design.md.

15. Future Work

- Train the Johnson model on a larger and more appropriate image dataset.
- Evaluate a broader collection of content and style images.
- Introduce perceptual evaluation methods and human evaluation.
- Evaluate AdaIN alpha values across multiple content and style image pairs.
- Benchmark CPU and additional GPU hardware.
- Evaluate batch inference and larger resolutions.
- Investigate video style transfer and temporal consistency.
- Explore model compression and reduced-precision inference.
- Build an interactive style transfer interface.
- Automate the complete experiment suite and standardized reporting.

16. References

Gatys, L. A., Ecker, A. S., & Bethge, M. (2015). A Neural Algorithm of Artistic Style.
<https://arxiv.org/abs/1508.06576>

Johnson, J., Alahi, A., & Fei-Fei, L. (2016). Perceptual Losses for Real-Time Style Transfer and Super-Resolution.
<https://arxiv.org/abs/1603.08155>

Huang, X., & Belongie, S. (2017). Arbitrary Style Transfer in Real-time with Adaptive Instance Normalization.
<https://arxiv.org/abs/1703.06868>

Simonyan, K., & Zisserman, A. (2014). Very Deep Convolutional Networks for Large-Scale Image Recognition.
<https://arxiv.org/pdf/1409.1556>

Studio NST is an experimental research implementation. Results are specific to the documented configurations and hardware environment.